EECS 221: Machine Learning for Science

Data–Compute Trade-offs in Neural PDE Forecasting

Suha Kagan Kose

Mid-Project Report

Project goal. Neural PDE forecasting models must be useful under limited data and compute. We compare how accurately and efficiently three architecture families with different inductive biases (a convolutional U-Net, a Fourier neural operator and a vision transformer) learn Rayleigh–Bénard convection when training data and GPU time are limited. This check-in covers the most constrained setting of the study: 10% of the training data and 1 or 4 hours of training on one A100 GPU. The 100% runs follow (Section 7).

Milestones. The abstract set three milestones for this check-in; all three are met.

1. **Data pipeline validated against The Well's released checkpoints.** Our data path and metric reproduce the published test error of the released CNextU-Net and TFNO to within 1% (Section 2).
2. **Budgets fixed from pilot timings.** Batch 64, fp32 and a peak learning rate of 10^{-3} for every model, from an equal-budget search; budgets count active GPU training time (Section 3).
3. **First 10%-data results and a revised plan.** Six runs (3 models × 2 seeds) evaluated on the full test set at 1 and 4 hours, with their cost to train and to use (Sections 4–7).

1 Setup

Data and task. `rayleigh_benard` from The Well [1]: 1,750 simulated trajectories of 200 frames on a 512×128 grid (periodic in x, walls in y), across 35 (Rayleigh, Prandtl) regimes, with the official 80/10/10 splits. From four frames of buoyancy b, pressure p and velocity (u, w), a model predicts the next frame, 0.25 simulation time units later [12], and forecasts further by feeding its predictions back in (autoregressive rollout). This report trains on a nested 10% subset: 140 trajectories, four per regime. Windows from one trajectory are strongly correlated, so this is closer to 140 independent samples than to its 27,440 training windows.

Models. The Well's implementations, trained from random initialisation with matched parameter counts and GELU activations (Table 1). They differ in how they move information across the domain, which is what this study tests.

Training. AdamW, one-step MSE on normalised fields, batch 64, fp32, peak learning rate 10^{-3} with 500 warmup steps and cosine decay to zero over four hours of active GPU training. The one-hour results are the same runs' checkpoints after one hour (learning rate still at 85% of its peak). Two seeds per model change the initialisation and data order; all runs share one 10% subset. The paper's recipe differs in learning rate, schedule and hardware (Appendix A).

Evaluation. The error measure is The Well's VRMSE: per field f and predicted frame,

$$\mathrm{VRMSE}_f = \sqrt{\frac{\frac{1}{N}\sum_i (\hat{y}_{f,i} - y_{f,i})^2}{s_f^2 + \varepsilon}}, \qquad s_f^2 = \text{spatial variance of the target field}, \quad \varepsilon = 10^{-7},$$

Model	Params	How it mixes information	Inductive bias
CNextU-Net [2]	18.6M	ConvNeXt convolution blocks at four resolutions, joined by skip connections	local and multi-scale
TFNO [3, 4]	19.3M	global convolution through the 16×16 lowest Fourier modes (factorised weights), plus pointwise layers	global and smooth: finer scales are not mixed in space
AViT [5]	18.6M	16×16-pixel patches as tokens, attention along each axis	global between patches; resolution set by the patch

Table 1: The three families. AViT's width was raised from 384 to 432 to match parameter counts.

averaged over the four fields. A value of 1 means an error as large as the field's own variability. We score the fields as stored (*physical-field VRMSE*) and, to compare with the paper, also on normalised fields (*paper protocol*); the two differ only through ε (Appendix C). One-step error covers all 196 windows of the 175 test trajectories, split into *onset* (first input frame < 50) and *developed* convection. Rollouts start at frames 20, 80 and 140 and run 30 steps (7.5 time units); we report the error and the fraction of rollouts that stay below VRMSE 1, 2 and 5. Horizontal kinetic-energy spectra show which scales are right. The reference is *persistence*, which copies the last input frame. Inference time is measured at batch 1 (one forecast) and per sample at batch 64 (many forecasts at once).

2 Milestone 1: Pipeline Validated Against the Released Checkpoints

We scored The Well's released CNextU-Net and TFNO with our own data path and metric on every test window (Table 2). Both reproduce the paper's one-step test values to within 1%, and their rollouts land in the paper's reported range. The released TFNO had to be run with the forward pass it was trained with: the paper-era code skipped its Fourier layers, an error fixed upstream in 2025 (Appendix B).

	Paper-protocol one-step VRMSE		Rollout from frame 0, steps 6–12 / 13–30	
Model	Ours (released weights)	Published	Ours	Published
CNextU-Net	0.669	0.670	36 / 35	> 10 / > 10
TFNO	0.663	0.657	142 / 169	> 10 / > 10

Table 2: Released checkpoints scored by our pipeline against the paper's Tables 2 and 3 (test split).

The check also showed what the headline number hides (Fig. 1). During the first $\sim$50 frames the fluid is nearly at rest: the velocity's spatial variance, the VRMSE denominator, is close to zero, so small absolute errors become relative errors of 1–10, while persistence is almost exact. From frame 50 on, the released CNextU-Net scores 0.038, against 0.669 averaged over all windows. We therefore report onset and developed convection separately.

3 Milestone 2: Budgets Fixed from Pilot Timings

Budget clock. The cluster's shared storage delivers at most $\sim$98 training samples per second into one node and stalls for minutes at a time, while TFNO and AViT consume 116 and 330 per second of GPU time. A wall-clock budget would partly measure the storage, so the 1- and 4-hour budgets count *active GPU training time*; data waits and the GPU time each job held are reported separately (Appendix D).

Learning rate and batch. An equal-budget search (20 minutes of training per run, validation only) over $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}\}$ chose 10^{-3} for every model (Appendix E). It is the top of the grid, so it is the

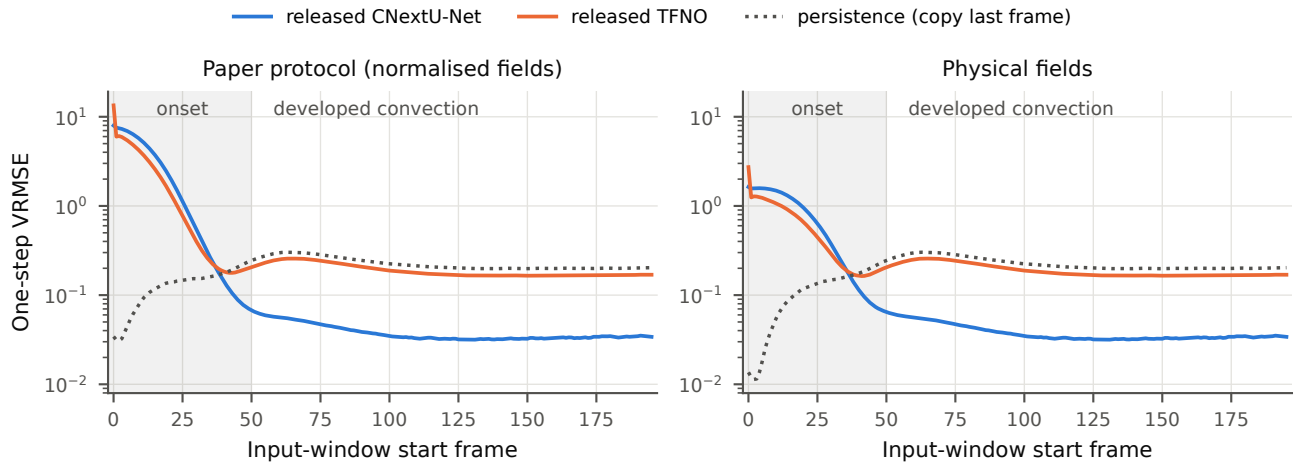


Figure 1: Released checkpoints' one-step VRMSE by window on the test set (mean over 175 trajectories), on normalised (left) and physical fields (right). Shaded: *onset* windows, first input frame below 50, a cut chosen where the error levels off on the validation split. Persistence copies the last input frame.

best rate tested rather than an established optimum. Batch 64, as in The Well, fits every model in fp32 on an 80 GB A100. TFNO's FFTs require fp32 in this implementation, so all models train in fp32; this favours TFNO, since bf16 would speed up a training step 1.7 times for CNextU-Net and 2.2 times for AViT.

4 Milestone 3: Results at 10% Data

One-step accuracy (Table 3). After four hours, CNextU-Net is clearly the most accurate in developed convection (0.050, against 0.075 for AViT and 0.109 for TFNO), while AViT is best during onset and, narrowly, over all windows. Seeds agree to within 0.006. Persistence beats every model during onset but is 2–4.5 times worse in developed flow. The released CNextU-Net, trained on all the data, is better in developed flow (0.038), but it also differs in schedule and hardware: by our estimate it completed at most ∼25 of its 200 scheduled epochs, so its learning rate never annealed. After one hour the order changes: AViT leads over all windows (0.39) and CNextU-Net is last (0.73) and the most seed-dependent.

	Physical-field one-step VRMSE			Paper protocol
Model (10% data, checkpoint)	all windows	onset	developed	(all windows)
CNextU-Net, 4 h	0.136 ± 0.006	0.387 ± 0.025	**0.050** ± 0.000	0.349 ± 0.022
TFNO, 4 h	0.209 ± 0.001	0.502 ± 0.003	0.109 ± 0.000	0.488 ± 0.001
AViT, 4 h	**0.134** ± 0.001	**0.308** ± 0.003	0.075 ± 0.000	**0.298** ± 0.002
CNextU-Net, 1 h	0.728 ± 0.176	2.618 ± 0.681	**0.081** ± 0.003	2.345 ± 0.600
TFNO, 1 h	0.473 ± 0.078	1.505 ± 0.305	0.120 ± 0.000	1.362 ± 0.249
AViT, 1 h	**0.394** ± 0.036	**1.299** ± 0.140	0.085 ± 0.000	**1.171** ± 0.125
Released CNextU-Net (100%, paper)	0.224	0.766	0.038	0.669
Released TFNO (100%, paper)	0.300	0.623	0.190	0.663
Persistence	0.199	0.121	0.226	0.202

Table 3: One-step test error, mean ± standard deviation over two seeds (0.000: below 0.0005). Bold: best of our models at each budget. The released TFNO lacks its Fourier layers (Appendix B).

Rollouts (Table 4, Fig. 2). One-step accuracy does not carry over in the same order. After four

hours CNextU-Net is the most stable: its error after 30 steps from developed flow (0.53, 0.40) is below persistence's (0.86, 0.54), and 89% of its rollouts stay below VRMSE 1. AViT, despite its one-step error, degrades fastest; after 30 steps from developed flow it is no better than persistence (0.72 against 0.70). After one hour TFNO is the most stable (60% of rollouts below VRMSE 1, against 34% for the others), and no model beats persistence after 30 steps. Persistence keeps the most rollouts below the thresholds only because its error levels off near 0.7; over the first steps every model is 2–5 times more accurate (Fig. 2a).

Model	VRMSE after 30 steps from frame 20 / 80 / 140	Fraction staying below threshold through step 30: 1 / 2 / 5	Paper protocol, steps 6–12 / 13–30 (normalised)
CNextU-Net, 4 h	1.78 / **0.53** / **0.40**	**0.89** / **0.94** / **0.98**	**15** / **30**
TFNO, 4 h	1.79 / 0.59 / 0.45	0.75 / 0.85 / 0.91	51 / 77
AViT, 4 h	5.42 / 0.80 / 0.65	0.63 / 0.80 / 0.87	75 / 167
CNextU-Net, 1 h	3.60 / 1.09 / 1.04	0.34 / 0.74 / 0.79	202 / 148
TFNO, 1 h	**2.67** / **0.90** / **0.78**	**0.60** / **0.77** / **0.86**	**81** / **116**
AViT, 1 h	4.67 / 1.11 / 0.98	0.34 / 0.74 / 0.83	129 / 184
Released CNextU-Net (100%)			36 / 35
Released TFNO (100%)			142 / 169
Persistence	0.68 / 0.86 / 0.54	0.94 / 1.00 / 1.00	0.27 / 0.50

Table 4: Rollouts on the test set (physical fields unless stated; mean over two seeds; 525 rollouts per model). Right: the paper's Table 3 protocol, rollouts from frame 0 (published: > 10). Bold: best of our models at each budget.

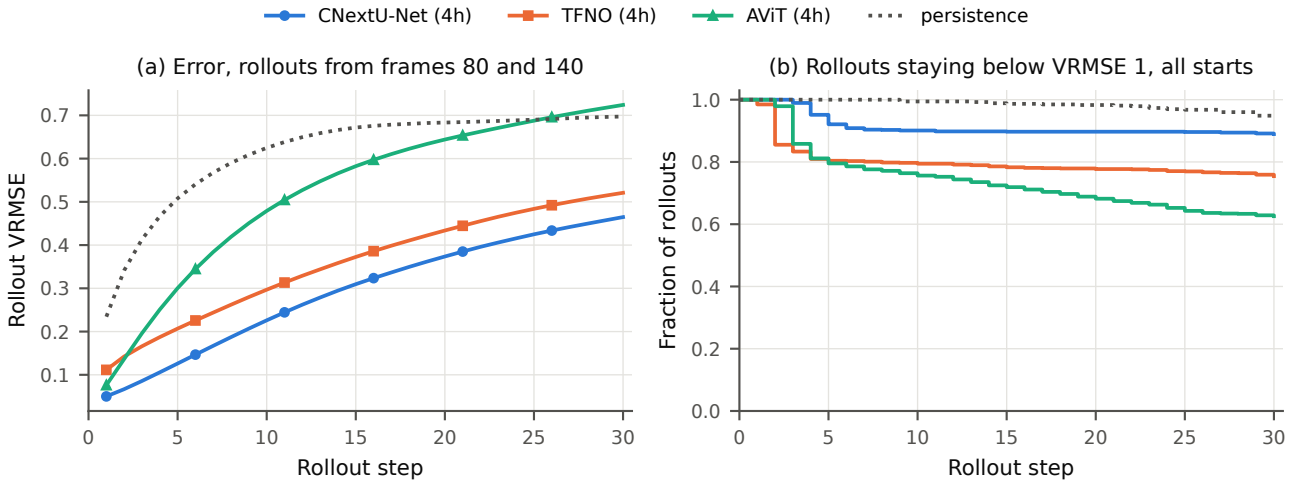


Figure 2: Four-hour models and persistence. (a) Rollout VRMSE per step from frames 80 and 140 (developed convection). (b) Fraction of all 525 rollouts still below VRMSE 1 at each step (Kaplan–Meier survival).

Accuracy against cost (Fig. 3, Table 5). From one to four hours the developed-flow error falls by 38% for CNextU-Net, 12% for AViT and 9% for TFNO. To *train*, AViT is the most efficient on the GPU (six times as many optimizer steps as CNextU-Net, a seventh of the memory), but it reads data faster than our storage delivers, so each four-hour run held a GPU for 11–12 hours, against 5–6 for CNextU-Net. To *use*, every model forecasts 30 steps in at most a quarter of a second with under 0.5 GB of memory. For one forecast at a time TFNO is the fastest (135 ms) and CNextU-Net the slowest (249 ms); for many forecasts at once AViT is the cheapest per sample (0.89 ms per step at batch 64, against 3.15 for TFNO and 6.74 for CNextU-Net).

Physical fidelity (Figs. 4 and 5). Per wavenumber, the median ratio of predicted to true kinetic energy over the 35 regimes shows where each model is wrong. CNextU-Net stays within 0.80–1.57 below $k_x = 100$.

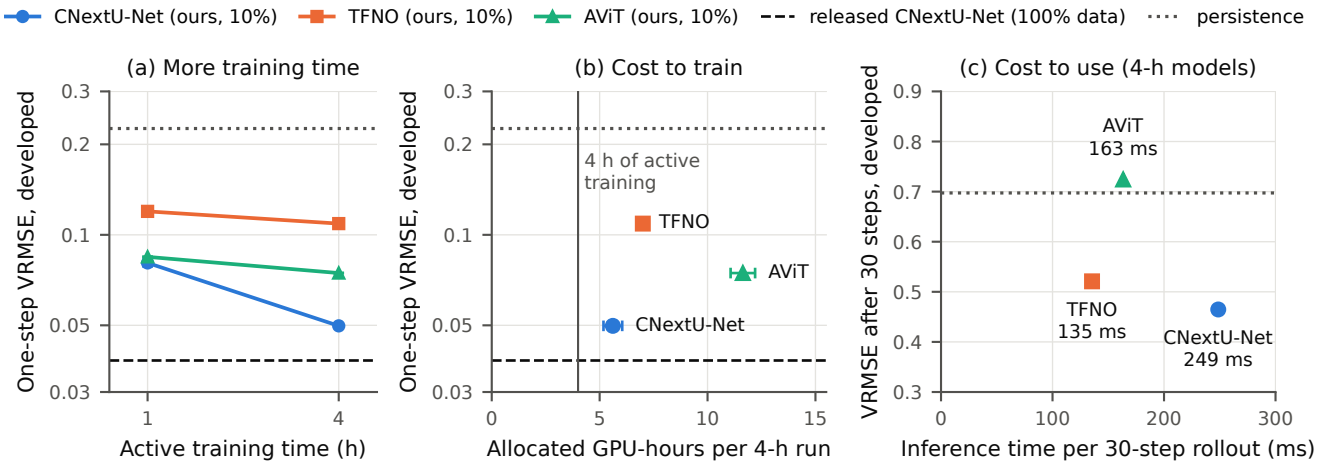


Figure 3: Accuracy against cost, 10% data (seed mean; bars: seed range). (a) Developed-flow one-step VRMSE at one and four hours. (b) The same four-hour error against the GPU time each run held, including data waits. (c) Error after 30 steps from frames 80 and 140 against the time for one 30-step forecast (batch 1). Dashed: released CNextU-Net (100% data); dotted: persistence.

	Training (4-hour budget, batch 64)				Inference			
Model	Optimizer steps	Epochs	Peak memory, alloc. (reserved)	GPU time held per run	per step, batch 1	30 steps, batch 1	per sample, batch 64	Memory, batch 1
CNextU-Net	12,122–12,194	28	60.9 (64.8) GB	5.2–6.0 h	8.3 ms	249 ms	6.74 ms	0.31 GB
TFNO	26,339–26,352	61	48.6 (56.9) GB	6.9–7.1 h	4.4 ms	135 ms	3.15 ms	0.44 GB
AViT	74,978–75,060	175	8.8 (8.9) GB	11.1–12.2 h	5.4 ms	163 ms	0.89 ms	0.23 GB

Table 5: Cost to train and to use on one A100 80 GB, fp32. GPU time held includes data waits, start-up, validation and checkpoints. Per sample at batch 64: forward pass in the training profile, an upper bound for inference.

TFNO is too smooth: between $k_x \approx 25$ and 100 its ratio falls to 0.33 after one step and 0.09 after 30. AViT is close after one step but gains energy over a rollout (0.55–21 below $k_x = 100$ after 30 steps), and both CNextU-Net and AViT put far too much energy in the smallest scales, where the true energy is tiny. The snapshots agree: CNextU-Net follows the plumes through 30 steps, TFNO blurs them and builds a warm layer along the bottom wall, and AViT shows patch-sized rectangular artifacts by step 12.

5 Why the Families Behave Differently

The results follow from how each architecture represents the flow, and they match known behaviour of neural PDE surrogates.

Spectral truncation sets TFNO's ceiling. A Fourier layer mixes information over the whole domain, but only through the lowest modes, here 16 in each direction ($k_x \lesssim 25$); finer scales pass only through pointwise layers [3, 4]. This bias suits smooth, large-scale dynamics, consistent with TFNO's early lead in rollouts after one hour. It also sets a floor that more training does not lower: TFNO's training loss stops falling after about 1.5 hours, its developed-flow error improves by only 9% from one to four hours, and it loses most of the energy between $k_x \approx 25$ and 100, blurring the thin plumes. More modes, or a hybrid with local convolutions, would be the usual remedies [6].

Locality at several resolutions suits convection, once CNextU-Net is trained. Convection is driven by local structures (thermal boundary layers, rising and falling plumes) organised into large rolls, which matches a U-Net's local convolutions at four resolutions. U-Net variants are among the strongest

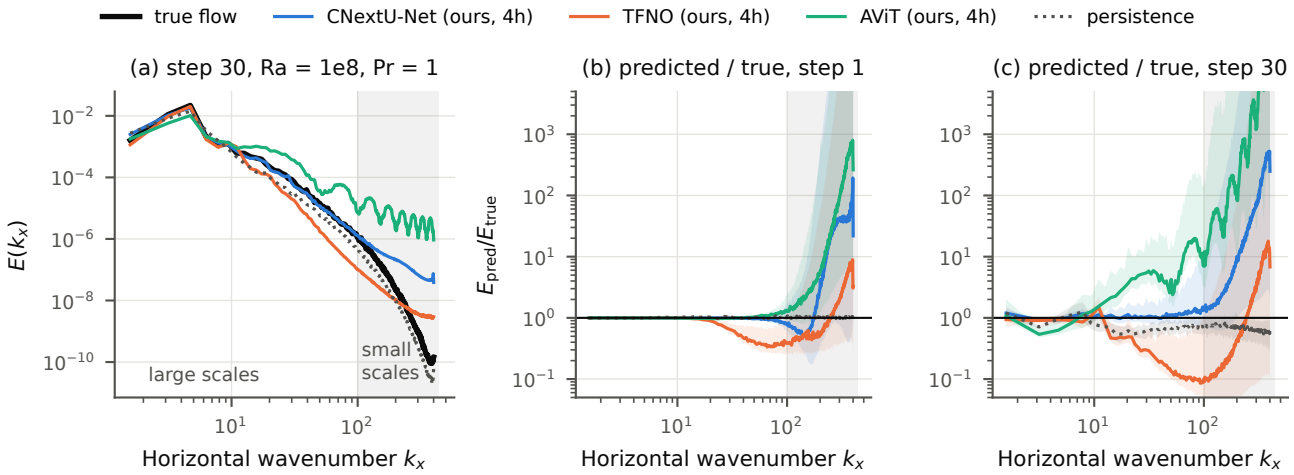


Figure 4: Horizontal kinetic-energy spectra of the four-hour models. (a) One regime ($\text{Ra} = 10^8$, $\text{Pr} = 1$) after 30 steps. (b, c) Predicted over true energy after 1 and 30 steps: median over the 35 regimes (band: middle 50%); 1 is exact. Shaded: small scales ($k_x > 100$).

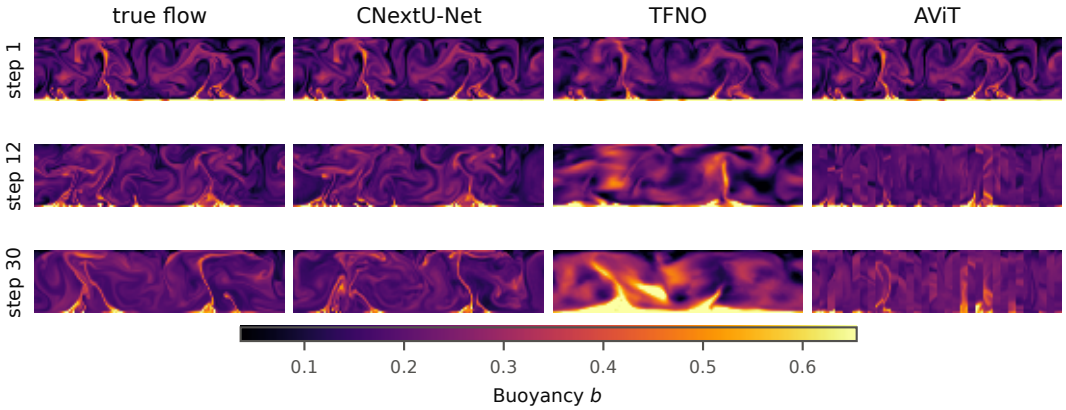


Figure 5: Buoyancy at $\text{Ra} = 10^8$, $\text{Pr} = 1$, rolled out from frame 80 by our four-hour models (seed 0) after 1, 12 and 30 steps. One colour range for all panels. $\text{Ra} = 10^{10}$ in Appendix E.

baselines in neural PDE benchmarks [6, 1]. CNextU-Net is also the most expensive per step (61 GB, 12,000 optimizer steps in four hours against AViT's 75,000), so it needs the longest training to pay off: least reliable after one hour, best after four, and still improving fastest.

The patch sets AViT's resolution. AViT turns 16×16-pixel patches into tokens and mixes them by attention [5]. That is cheap (six times as many steps per hour) and gives the best one-step error over all windows, but detail inside a patch is decoded patch by patch. Errors at the patch scale, visible as rectangular artifacts and as excess small-scale energy, then grow over a rollout.

One-step training does not guarantee stable rollouts. All three models minimise the one-step MSE. That loss weights each scale by its energy, so low-energy small scales are barely constrained (the spectral bias of neural networks [7]), and surrogates trained this way are known to neglect non-dominant, often high-frequency, components [8]. In a rollout the model then receives its own slightly wrong outputs, inputs it never saw in training [9], and uncontrolled growth of such errors is a known route to instability [10]. This is why one-step rankings do not carry over to rollouts, and why the excess small-scale energy of CNextU-Net and AViT matters. Training on the model's own outputs (the pushforward trick [9]), injecting noise or filtering high frequencies address it; they are outside this study.

Persistence and the metric. With frames 0.25 time units apart, the flow changes little per step, so

copying the last frame is a strong baseline. VRMSE is relative to the target's spatial variance, which is near zero during onset, so small absolute errors there become large relative ones. A useful model must beat persistence where the flow evolves; ours do so by 2–4.5 times per step in developed convection, but not during onset.

Data or compute? After 28 to 175 epochs over 140 trajectories, every model's validation error is still falling at the end of training (Appendix E), so none shows signs of overfitting within four hours: at this budget, compute rather than data appears to be the binding constraint. The 100% runs will test this directly.

6 Findings Under Limited Data and Compute

These hold for these configurations, 10% of the data and the tested budgets (Table 6).

- **Accuracy.** After four hours CNextU-Net is the most accurate in developed convection (0.050) and the most stable over rollouts (89% stay below VRMSE 1); AViT matches it over all windows but degrades fastest; TFNO is too smooth.
- **One hour.** AViT has the lowest one-step error and TFNO the most stable rollouts; CNextU-Net is the least reliable, but gains most from more training (38% against 12% and 9%).
- **Cost.** A four-hour run held a GPU for 5–6 (CNextU-Net), 7 (TFNO) or 11–12 hours (AViT, storage-bound). A single 30-step forecast takes 135–249 ms, fastest with TFNO; in batches AViT is the cheapest per forecast.
- **Negative results.** No model beats persistence during onset; after 30 steps from developed flow AViT is no better than persistence; TFNO's error stops improving early.
- **Caveats.** Two seeds on one fixed subset; fp32 favours TFNO; the learning rate comes from a short search.

If the priority is	Best of the three	Trade-off
Accurate forecasts of developed convection that stay stable over 30 steps	CNextU-Net	Slowest single forecast (249 ms per 30 steps); 61 GB training memory at batch 64; needs the longest training
Fastest single forecast (batch 1)	TFNO (135 ms per 30 steps)	Too smooth (loses energy between $k_x \approx 25$ and 100); stops improving early
Least training memory, cheapest batched forecasts	AViT (8.8 GB; 0.89 ms per sample at batch 64)	Needs storage delivering ∼330 samples/s; rollouts degrade fastest
Only one hour of training	AViT (one step); TFNO (rollouts)	CNextU-Net varies most between seeds; none beats persistence after 30 steps
Convection onset (first 50 frames)	None: persistence	Every model is less accurate than copying the last frame

Table 6: Which model to use under which constraint, from the 10% results after four hours unless stated.

7 Revised Plan

The core comparison is unchanged: three families, 10% and 100% of the training data, two seeds, checkpoints at one and four hours. The 100% runs complete the data axis: repeating Fig. 3 and Table 6 with

all the data will measure what ten times more data buys at each budget, and whether CNextU-Net's lead and TFNO's ceiling persist. Changes from the abstract:

- **Budgets** count active GPU training time; GPU time held, data waits and cost are reported separately.
- **Evaluation** separates onset from developed convection, reports all three failure thresholds, and adds the paper protocol for reference.
- **Resources.** The core study needs about 135 allocated GPU-hours instead of the abstract's 100, because of data waits (Appendix D). The 10% work used 72.6 GPU-hours (2,160 SU billed).
- **Optional**, after the core: a Dedalus [11] solver baseline, to set the models' forecast time and accuracy against the numerical solver's; a second draw of the 10% subset; 24-hour runs with the paper's learning rates.

Next steps. (1) Make data loading robust to storage stalls and test node-local staging; (2) run the six 100% runs and the data–compute comparison; (3) complete field-wise VRMSE and the Nusselt-number error. The abstract's extensions (input-noise robustness, a third seed, 30% data, ConvLSTM) follow only after the core.

Appendix A: Training Recipe

Reference settings come from the paper and The Well's configuration at commit `e63dec8`. Our software: The Well `e63dec8`, neuraloperator 0.3.0, PyTorch 2.8.0 (CUDA 12.8), Python 3.13.

	This study	The Well paper / code
Activation	GELU (all three models)	GELU (same model code)
Optimizer	AdamW, $\beta = (0.9, 0.999)$, weight decay 10^{-4}, no clipping	AdamW, weight decay 10^{-4} in the code (0.01 in the text), no clipping
Loss	one-step MSE, normalised units, equal field weights	same
Batch size, precision	64, fp32	64, fp32
Learning rate	10^{-3} for all, from our search over $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}\}$	per model from $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}, 5\cdot10^{-3}, 10^{-2}\}$: $5\cdot10^{-4}$ (CNextU-Net), 10^{-4} (TFNO)
Schedule	500 warmup steps, cosine to 0 over the 4-hour budget	5 warmup epochs, cosine to 0.1·lr over 200 epochs, stopped at 12 h
Budget, hardware	1 h and 4 h of active GPU training on one A100	12 h on one H100, best validation epoch
Training data	10% of the trajectories (100% next)	100%

Table 7: Training recipe: ours against the paper's.

Released CNextU-Net's schedule. Our A100 trains CNextU-Net at 54 samples per second; with an H100 at most three times faster, 12 hours give at most ∼7 million samples, about 25 epochs of 274,400 windows. With 5 warmup epochs and a cosine over 200, the learning rate at epoch 25 is still above 95% of its peak. This is an estimate; the model card does not give the epoch count.

Appendix B: TFNO Forward Pass

In The Well's code, commit `2d8d7eaf` (2024-06-03) called TFNO's Fourier blocks without keeping their output, so the model computed only its lifting and projection layers, a per-pixel network; commit `e4301602` (2025-11-17) restored the assignment [13]. The paper's TFNO results and the released checkpoint date from that period. Scored with the forward pass it was trained with, the released TFNO reproduces the published value; with the corrected forward pass the same weights score ≈10.5. Our runs use the fixed code. Upstream issue #49 reports related inconsistencies [14].

Appendix C: Metric Details

VRMSE. The Well's `VRMSE` class at `e63dec8`, over the $N = 512 \times 128$ points of a target frame, with s_f^2 the unbiased spatial variance; the four fields are averaged with equal weights and windows with equal weight. The paper protocol scores normalised fields $(y_f - \mu_f)/\sigma_f$ with The Well's training statistics ($\sigma_f = 0.249, 0.149, 0.208, 0.180$ for b, p, u, w). Without ε the two would be identical, since the ratio is invariant to an affine rescaling; with ε, normalising shrinks the floor from 10^{-7} to the equivalent of $\varepsilon\sigma_f^2 \approx 2\text{–}6 \times 10^{-9}$. During onset the velocity is nearly zero (exactly zero at $t = 0$) and its spatial variance is comparable to 10^{-7}, so the physical-field score is damped there and the normalised one is not. This is the whole difference between the released CNextU-Net's 0.669 (paper protocol) and 0.224 (physical fields): on onset windows they score 2.510 and 0.766, while from frame 50 on both give 0.038.

Failure horizon. The first step whose field-mean VRMSE exceeds a threshold (1, 2 or 5). With no rollout ending before step 30, the Kaplan–Meier survival at step 30 equals the fraction of rollouts staying below the threshold.

Kinetic-energy spectrum. At each grid row the velocity is Fourier-transformed along the periodic x axis; $E(k_x) = \frac{1}{2}(|\hat{u}|^2 + |\hat{w}|^2)$ (one-sided) at $k_x = 2\pi n/L_x$ with $L_x = 4$, averaged over the Chebyshev-spaced rows with quadrature weights, so that summing over k_x gives the mean kinetic energy. Spectra are averaged over each regime's 15 rollouts (5 trajectories × 3 starts) at steps 1, 10 and 30. The spectrum describes how energy is spread over scales, not where structures are.

Inference time. Batch 1, fp32, input already on the GPU: after 10 warm-up predictions, the mean of 50 one-step predictions and one full 30-step rollout, each ended by a CUDA synchronisation. At batch 1 the GPU is far from fully used, so latency includes per-call overhead.

Appendix D: Resource Accounting

Timer. Active training time is measured per optimizer step with Python's `perf_counter`, from the moment the data loader returns the batch to just after a CUDA synchronisation following the optimizer update. It excludes waiting for data, validation, checkpoint writes and start-up. After a crash the clock resumes from the last checkpoint, so lost steps are not counted.

Validation sample. 64 fixed windows evenly spaced over the validation split's 34,300 windows, covering all 35 regimes; 28% are onset windows. It monitors training and does not replace the test evaluation.

Billing. The paid partition charges 32 SU per GPU-hour plus 1 SU per core-hour (40 SU per hour for one A100 and 8 cores); the free partition is not billed. Of the 72.6 allocated GPU-hours so far, 54.2 were on the paid partition (2,160 SU) and 18.3 on the free one.

Hours per run	CNextU-Net	TFNO	AViT
10% runs, measured			
Queue wait before training started	0.1–0.9	0.5–0.7	6.7–8.8
GPU training (the budget)	4.0	4.0	4.0
Waiting for data	0.8–1.1	2.5–2.8	6.1–7.3
Start-up, and steps replayed after a preemption or crash	0.13–0.65	0.14–0.15	0.68–0.69
Validation (64 windows every 30 minutes)	0.01–0.02	0.02–0.10	0.18
Checkpoint writes	0.23–0.24	0.08–0.15	0.03–0.13
Allocated time of training	**5.2–6.0**	**6.9–7.1**	**11.1–12.2**
Test evaluation job (1-hour and 4-hour checkpoints)	0.87–0.95	0.57	0.34–0.38
SU per training run (paid partition)	189–191	274–286	443–488
100% runs, projected			
Waiting for data	1.0–2.0	3.0–4.5	9.5–12.5
Allocated time of training	**5.5–6.5**	**7.5–9**	**14–17**

Table 8: Time per run (range over the two seeds). CNextU-Net's first attempts ran on the free partition, so its SU covers 4.8 h of its 5.2–6.0 h. The 100% projection assumes the measured storage rate (∼98 samples/s) and that the full training split (∼290 GB) no longer fits in a node's page cache.

Component	Allocated GPU-hours
Six 10% runs, completed (24.0 of them active training)	49.8
Learning-rate search, profiling, pipeline check, environment checks, first replication try	18.5
Test evaluation of the six 10% runs and persistence (completed)	4.2
Six 100% runs (projected, 24.0 active)	∼54–65
Test evaluation of the 100% runs	∼4.5
Core study	∼131–142

Table 9: Allocated GPU-hours of the core study: completed parts from Slurm accounting, the rest projected. No contingency is included; the abstract's ∼100 hours included ∼30.

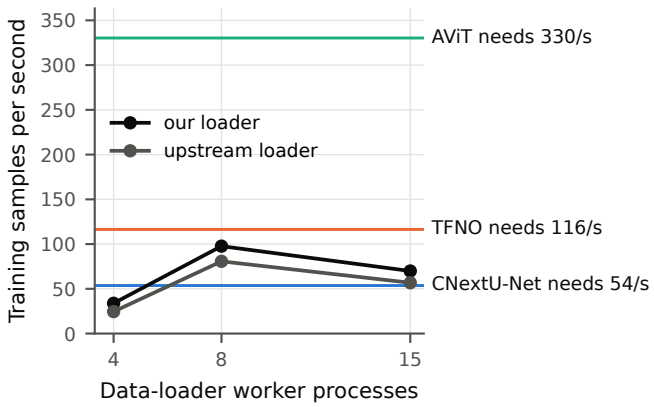


Figure 6: Training samples per second read from the shared file system by one node, against the rate at which each model consumes them on the GPU (horizontal lines).

Appendix E: Additional Results

Validation VRMSE after 20 min	10^{-4}	$5 \cdot 10^{-4}$	10^{-3}
CNextU-Net	0.841	0.469	**0.408**
TFNO	0.603	0.358	**0.302**
AViT	0.646	0.332	**0.257**

Table 10: Learning-rate search: 20 minutes of training per run, 10% data, 512 validation windows.

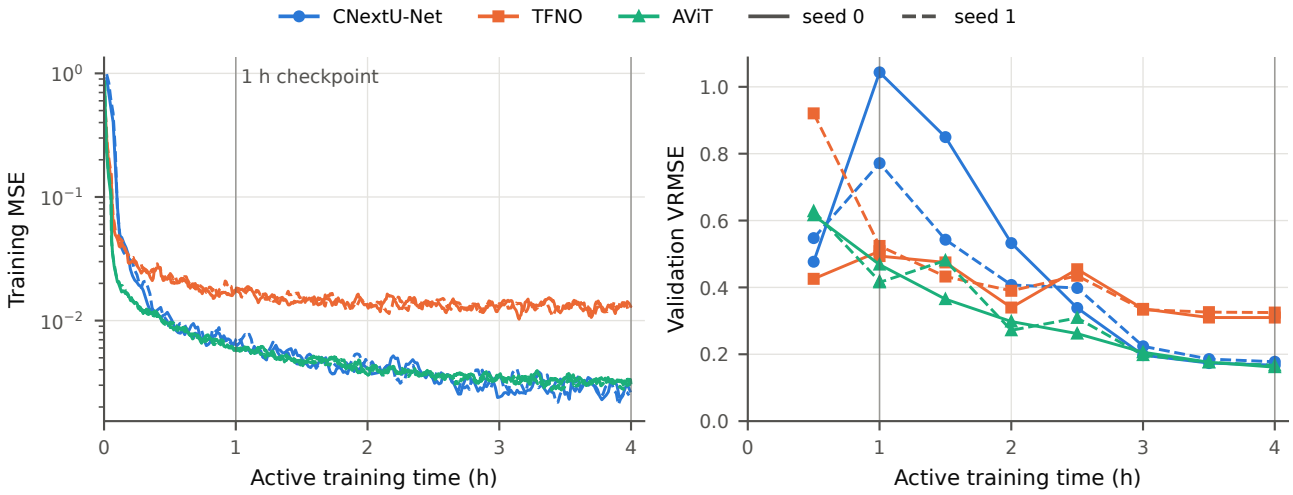


Figure 7: Training MSE (normalised units, smoothed over 3 minutes) and validation VRMSE on the 64 fixed windows (physical fields), against active training time, 10% data. Solid: seed 0; dashed: seed 1.

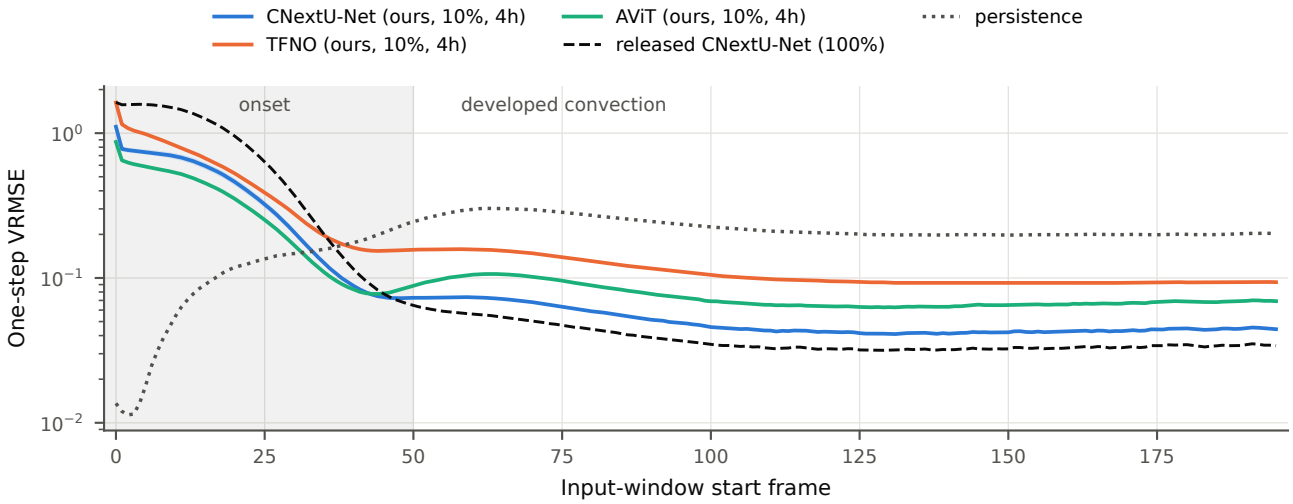


Figure 8: One-step test VRMSE (physical fields) by window: our four-hour models (seed mean; band: seed range), the released CNextU-Net trained on all the data, and persistence. Shaded: onset windows.

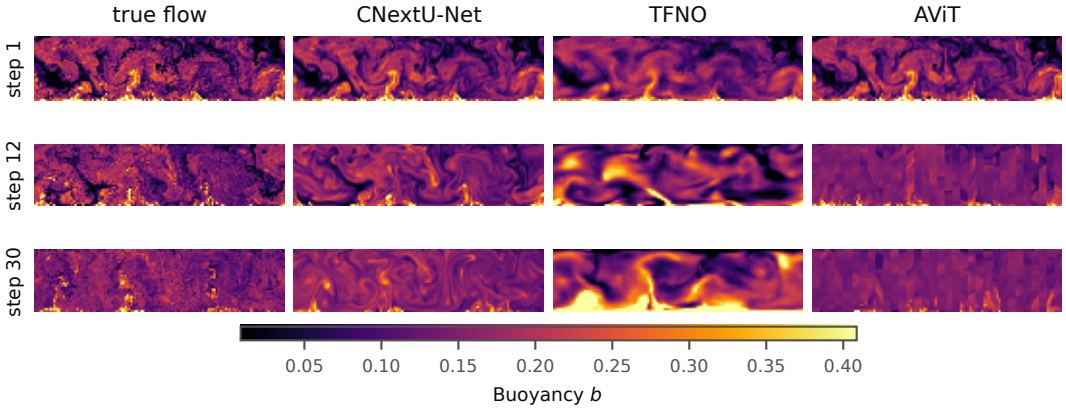


Figure 9: As Fig. 5, at Ra $= 10^{10}$: the true flow is far finer than any model reproduces; CNextU-Net keeps the large-scale plume pattern but smooths the small structures.

References

[1] Ohana, R., et al. "The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning." NeurIPS, Datasets and Benchmarks Track, 2024.
https://arxiv.org/abs/2412.00568

[2] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. "A ConvNet for the 2020s." CVPR, 2022.
https://arxiv.org/abs/2201.03545

[3] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. "Fourier Neural Operator for Parametric Partial Differential Equations." ICLR, 2021.
https://arxiv.org/abs/2010.08895

[4] Kossaifi, J., Kovachki, N., Azizzadenesheli, K., and Anandkumar, A. "Multi-Grid Tensorized Fourier Neural Operator for High-Resolution PDEs." TMLR, 2024.
https://arxiv.org/abs/2310.00120

[5] McCabe, M., et al. "Multiple Physics Pretraining for Spatiotemporal Surrogate Models." NeurIPS, 2024.
https://arxiv.org/abs/2310.02994

[6] Gupta, J. K., and Brandstetter, J. "Towards Multi-spatiotemporal-scale Generalized PDE Modeling." arXiv:2209.15616, 2022.
https://arxiv.org/abs/2209.15616

[7] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F. A., Bengio, Y., and Courville, A. "On the Spectral Bias of Neural Networks." ICML, 2019.
https://arxiv.org/abs/1806.08734

[8] Lippe, P., Veeling, B. S., Perdikaris, P., Turner, R. E., and Brandstetter, J. "PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers." NeurIPS, 2023.
https://arxiv.org/abs/2308.05732

[9] Brandstetter, J., Worrall, D., and Welling, M. "Message Passing Neural PDE Solvers." ICLR, 2022.
https://arxiv.org/abs/2202.03376

[10] McCabe, M., Harrington, P., Subramanian, S., and Brown, J. "Towards Stability of Autoregressive Neural Operators." TMLR, 2023.
https://arxiv.org/abs/2306.10619

[11] Burns, K. J., Vasil, G. M., Oishi, J. S., Lecoanet, D., and Brown, B. P. "Dedalus: A flexible framework for numerical simulations with spectral methods." Physical Review Research 2, 023068, 2020.
https://arxiv.org/abs/1905.10388

[12] The Well, `rayleigh_benard` dataset card, commit `e63dec8`.
https://github.com/PolymathicAI/the_well/blob/e63dec8/datasets/rayleigh_benard/README.md

[13] The Well source code, commits `2d8d7eaf9eb46b96f215abf39c94b6be4bf9ea5c` (2024-06-03) and `e4301602e57f2dddeaa87856dda49bbba47c663b` (2025-11-17).
https://github.com/PolymathicAI/the_well/commit/2d8d7eaf9eb46b96f215abf39c94b6be4b

f9ea5c
https://github.com/PolymathicAI/the_well/commit/e4301602e57f2dddeaa87856dda49bbba47c663b

[14] The Well, issue #49, "Inconsistent Pre-trained Baseline Performance" (open since 2025-04-21). https://github.com/PolymathicAI/the_well/issues/49